

Bridging AMD's Softpower Gap

**Why the best inference silicon doesn't win by itself — and
how the Physical AI SDK closes the gap**

**Executive Briefing · Voice AI as the lens — the same pattern repeats in robotics,
industrial & healthcare**

The Stakes — A Market Being Decided Right Now

Metric	Value
AI voice-agents market, 2034	\$47.5B @ 35% CAGR
Cost per call — human vs AI	\$7–12 → \$0.40
YoY growth in production deployments	340% (500+ orgs)
Contact-center labor savings, 2026	\$80B (Gartner)
Orgs planning voice AI by end-2026	80%

It is a compute market at its core

- A deployed agent is a **steady-state inference machine** — it doesn't train; it transcribes, reasons & speaks **24×7**
- At scale, its **dominant operating cost is GPU compute**
- **3 of the 5** pipeline layers — **ASR · LLM · TTS** — are GPU-compute-bearing

The compute substrate for a massive, fast-growing inference market is being **chosen right now** — and whoever owns the default owns the decade.

AMD Already Has the Winning Silicon

LLM inference — MI300X vs H100 (Llama 3.1 70B, 2026)

Dimension		H100 SXM5
VRAM	192 GB HBM3	80 GB
Cloud price	\$1.50–2.50/hr	\$2.90/hr
Throughput (tok/s)	~18,000	~19,500
Cost / 1M tokens	\$0.027	\$0.041
70B fits on 1 GPU?	Yes	No (needs 2)

The savings compound at scale

- 34% lower cost-per-token → ~\$84K/month saved at 1M min/month
- Self-hosted ROCm TTS removes \$100K–\$300K/month in hosted synthesis
- India on-prem: ~50% lower GPU CapEx
- MI355X within single-digit % of B200 — MLPerf Inference v6.0

This is not a hardware problem.

Yet AMD Is Absent Where the Decision Is Made

The real decision isn't "which GPU"

- Made **upstream and invisibly** — before a developer thinks about silicon
- NVIDIA's edge is **softpower**: a layered ecosystem surface that is the **path of least resistance** at the first touchpoint
- The instrument: the **NIM playbook**

*"Follow a Blueprint and you **never consciously select a GPU** — the choice is already made. Every other choice requires active effort."*

The NIM ecosystem — how the default gets set

NVIDIA NIM — Hardware as Platform

Software

Riva ASR · Maxine · ACE
Triton · TensorRT-LLM
AI Blueprints

Developer

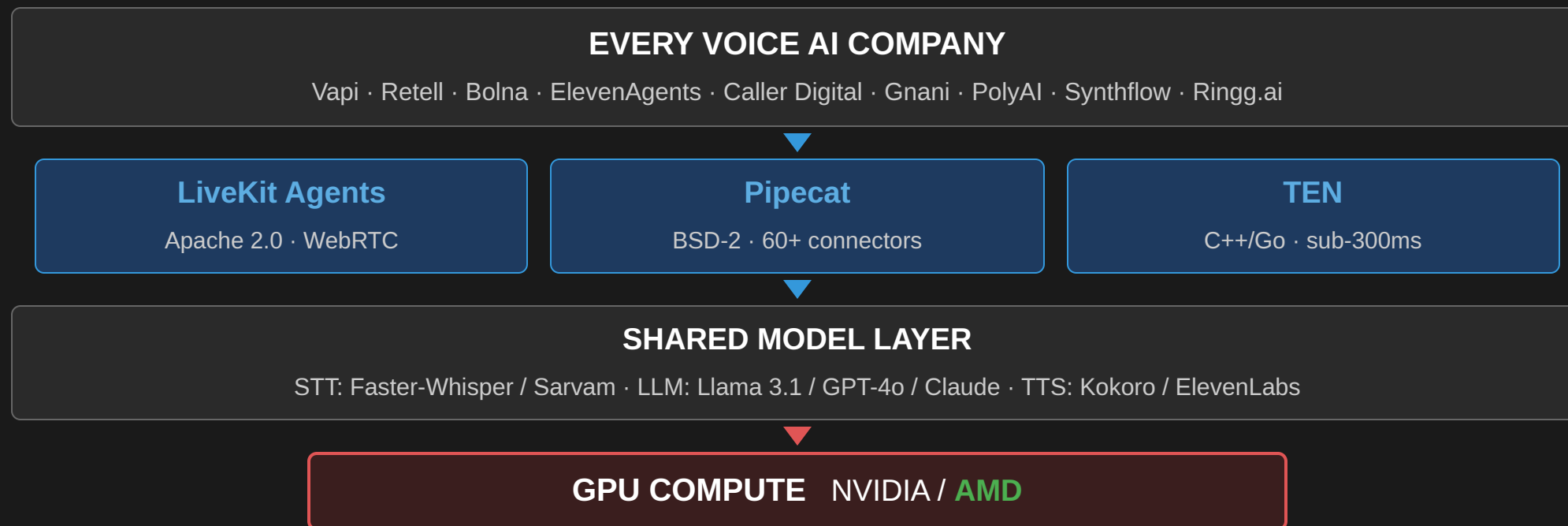
28M devs on CUDA
VS Code NIM ext.
RTX Spark devkit

Channel

Azure Foundry NIM
SI certs: TCS·Infosys
Wipro·Accenture

- `docker pull` → running in **30 seconds**
- `ChatNVIDIA(...)` → routed to NVIDIA **at the call site**
- Prototype on NVIDIA → **default to NVIDIA in production**

The Convergence — One Compute Decision Rules the Whole Market



Every commercial platform is a business layer above **one of three frameworks**, sharing a common model layer. **Win the compute layer — or the plugin slot just above it — and the hardware choice is made for every platform above.**

The Gap, Mapped — Five Downstream Layers, Four Upstream Barriers

5 downstream layers — visible & closeable

Layer	NVIDIA	AMD
Framework plugins	LangChain · LlamaIndex · Pipecat · LiveKit	—
Inference serving	Triton / Dynamo · TensorRT-LLM	vLLM ROCm only
Voice-domain SDKs	Riva · Maxine · ACE	—
Edge platform	Jetson · JetPack · Isaac	Ryzen AI (laptop only)
Cloud marketplace	Azure · HF · SageMaker · Vertex	absent / limited

4 upstream barriers — deeper, durable lock-in

- **Training / fine-tuning** — NeMo + CUDA-primary libs (bitsandbytes, trl, Axolotl, FlashAttention 3)
- **ISV & SI channel** — Genesys, Cisco, Salesforce; TCS/Infosys/Wipro certified on NVIDIA
- **Voice + vision convergence** — Jetson Thor unifies Isaac · Metropolis · Riva · ACE; AMD has no unified edge module
- **Synthetic data** for low-resource languages — no AMD-native pipeline or academic partnership

Closing the **downstream** layers puts AMD in contention; closing the **upstream** barriers creates **platform momentum**. The chain compounds: fine-tune on NeMo → deploy to Triton → shipped inside Genesys → sold by an SI certified on NIM.

The Sharpest Illustration — "Not a Silicon Gap, a Slot Gap"

HuggingFace Inference Endpoints

- AMD has a **formal hardware partnership** with HuggingFace
- The TGI ROCm image is **production-ready** — validated on MI210, MI250, MI300X (Flash Attention 2, AWQ, DeepSpeed)
- **The engineering is done.**

But at the point of decision...

- Open the "deploy this model" UI → the GPU dropdown lists **T4 → H100**
- **MI300X does not appear.**
- The developer picks NVIDIA — **not because AMD can't run the model, but because AMD isn't on the menu**

The gap in its simplest form: not a silicon gap — a slot gap. And a slot gap is closeable.

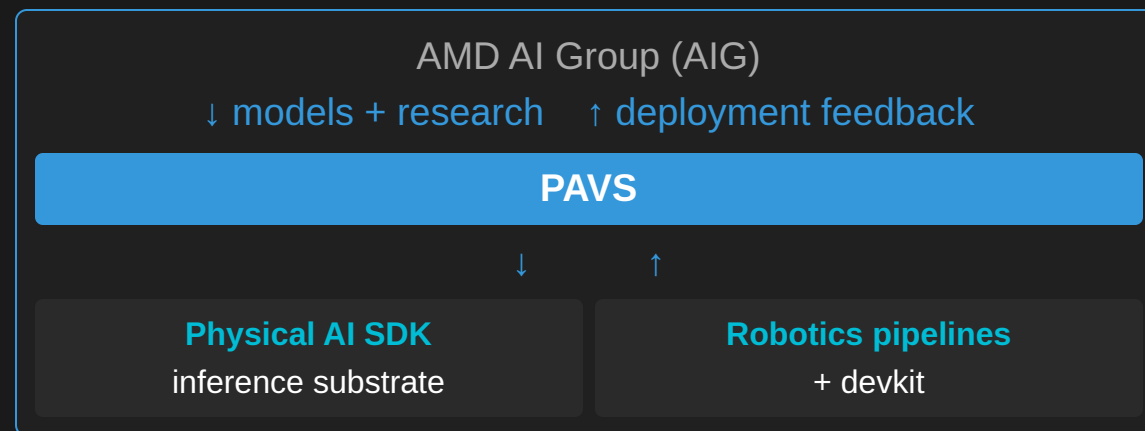
The Bridge Is Already Being Built

The gap can't be closed by a new chip

- It's a **software & ecosystem** problem — containers, plugins, blueprints, marketplace slots
- Which is **exactly** the kind of problem a **vertical-software team** solves

PAVS sits at the intersection

- Between **AIG model outputs** and **lighthouse customers** who validate & deploy in real environments
- Every conversion of a raw model → production vertical software does **organically** what NVIDIA's platform teams do **intentionally**



The assets it produces map 1:1 to the gap:

- Unified inference SDK → plugin + serving layers
- Multi-modal pipelines → voice + vision
- Jetson-class devkit → edge landing zone

PAVS DevKit — AMD's Jetson Moment

Capability	NVIDIA Jetson	
Form factor	Module (Nano → Thor)	Ryzen AI embedded APU
OS + SDK	JetPack (CUDA+Riva+Isaac)	Physical AI SDK (ROCm+Ryzen AI)
Model supply	NIM containers	AIG zoo → PAVS vertical SW
Runtime	TensorRT + Triton	vLLM · MIGraphX · llama.cpp · ONNX
Voice	Riva	Whisper + Kokoro + pipeline
Multi-modal	Isaac + Riva + ACE	VLM + VLA on one NPU/GPU

The architectural edge

- Ryzen AI APUs put **x86 CPU + RDNA GPU + XDNA NPU on one die** with **unified memory** — no CPU ↔ GPU copy overhead
- For **VLA models** (visual + language → robot action), NPU+GPU may beat Jetson's ARM+CUDA

The technical foundation — unified SDK, heterogeneous inference, robotics + voice pipelines, VLM/VLA support — is **being assembled by PAVS now**. What remains is the ISV/SI certification ecosystem.

How Physical AI SDK Maps to the Gap

NVIDIA ecosystem asset	Physical AI SDK answer
NIM certified containers — one docker pull	OOB + Docker — ROCm + RyzenAI pre-installed, reproducible
Framework plugin slots — endpoint-level default	Unified inference engine — one interface, CPU · GPU · NPU, every language
Domain SDKs / AI Blueprints (Riva, ACE)	Co-engineered vertical software — healthcare · automotive · industrial
Jetson + JetPack edge platform	Ryzen AI + ROCm edge enablement — HW patches, Yocto / LTS
NeMo fine-tuning lock-in	AIG generic → task-specific operationalization + feedback loop

Here's what that SDK actually is →

Physical AI SDK

The Low-Resistance Path to AMD Silicon

One Interface · Every Language · CPU · GPU · NPU

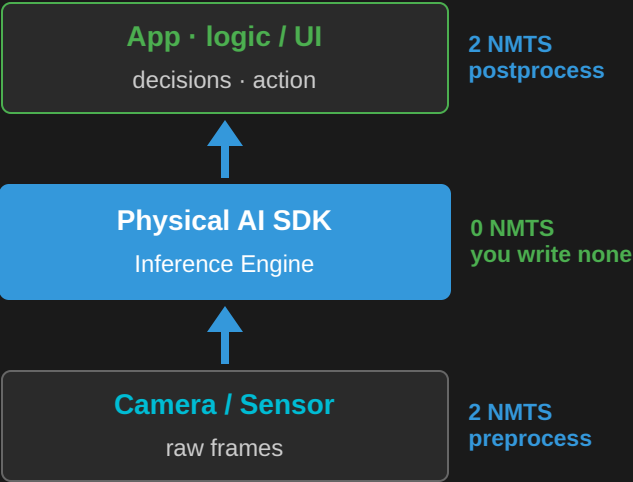
What is Physical AI SDK?

- **Physical AI SDK (PAI SDK)** is a **unified inference engine** that catalyses your product development on AMD hardware.
- It does this by creating a "**Low-Resistance Path**" to AMD silicon.
- A single **unified interface** to run AI workloads across **CPU, GPU, and NPU** on AMD edge hardware.

Learn one interface — deploy across every AMD compute device, from edge CPU to iGPU to NPU.

Physical AI SDK — The Pitch

From Camera to Product — in 4 NMTS



- **2 NMTS in** — capture, hand to the SDK
- **0 NMTS middle** — the SDK is the engine
- **2 NMTS out** — act on the results

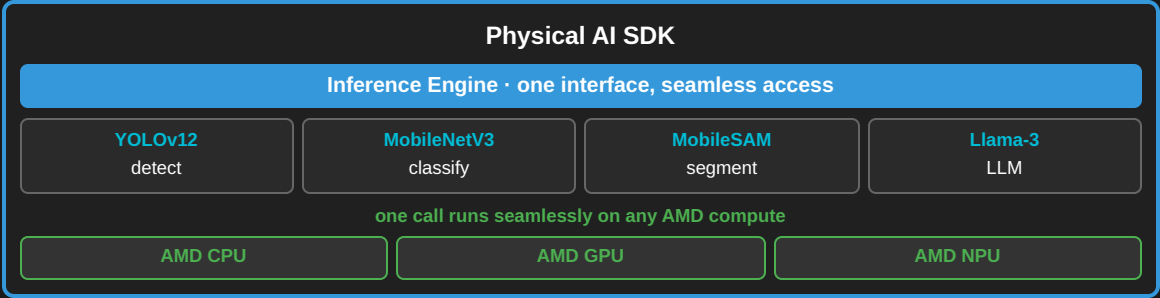
Total effort: 4 NMTS → a working edge app

NMTS = "Not More Than Ten (lines of code)" — one unit of developer effort.

The whole application — in Python

```
# NMTS 1 · set up the SDK
from physical_ai_sdk.inference_engine import InferenceEngine
engine = InferenceEngine("physical-ai-sdk.local")
yolo = engine.model("yolov12")
# NMTS 2 · read the camera
frame = Camera.open(0).read()
# NMTS 3 · inference — SDK does everything
dets = yolo.predict(frame)
# NMTS 4 · act on the results
for d in dets:
    draw_box(frame, d.box, d.label)
```

Same call, every language: [Python](#) [C++](#) [Kotlin](#) [JS](#)



Without PAI SDK — Triton + ROCm + gRPC setup, per-language rewrites, weeks to first inference

▶

With PAI SDK — one call, 4 NMTS, 2 engineers, half a week to a stable app on AMD

The Pitch: Physical AI SDK accelerates AI development on AMD Edge-HW — from a quarter to a week

Same Inference Call — Every Language

Python

```
from physical_ai_sdk.inference_engine \
    import InferenceEngine
engine = InferenceEngine(
    endpoint="physical-ai-sdk.local")
yolo = engine.model("yolov12")
dets = yolo.predict(frame)
```

C++

```
#include <physical_ai_sdk/inference_engine.hpp>
auto engine = InferenceEngine(
    "physical-ai-sdk.local");
auto yolo = engine.model("yolov12");
auto dets = yolo.predict(frame);
```

Android (Kotlin)

```
import com.amd.physical_ai_sdk.InferenceEngine
val engine = InferenceEngine(
    endpoint = "physical-ai-sdk.local")
val yolo = engine.model("yolov12")
val dets = yolo.predict(frame)
```

JavaScript / TypeScript

```
import { InferenceEngine }
    from "@amd/physical-ai-sdk";
const engine = new InferenceEngine({
    endpoint: "http://physical-ai-sdk.local:8000" });
const yolo = await engine.model("yolov12");
const dets = await yolo.predict(frame);
```

One mental model, four ecosystems. Same `Detection {label, score, box}` output everywhere.

What Am I Getting in This SDK?

The Engine

- **Unified inference engine** — one interface across **CPU · GPU · NPU**, every language (Python · C++ · Android · JS) and transport (in-place · gRPC · HTTP), **zero-copy on edge**

The Models

- **Optimized model zoo** — AMD-tuned models per task
- **Add a model = drop a config**; parallel · sequential · batched pipelines, all by config

The performance

- **Accuracy, throughput & PnP benchmarks** on edge for every model

The Delivery

- **OOB & Docker** — reproducible, ready-to-ship packaging
- **Co-engineering** with lighthouse customers — cut latency, tune accuracy

A **low-resistance path** to a **co-engineered, benchmarked, containerized** product on AMD edge hardware.

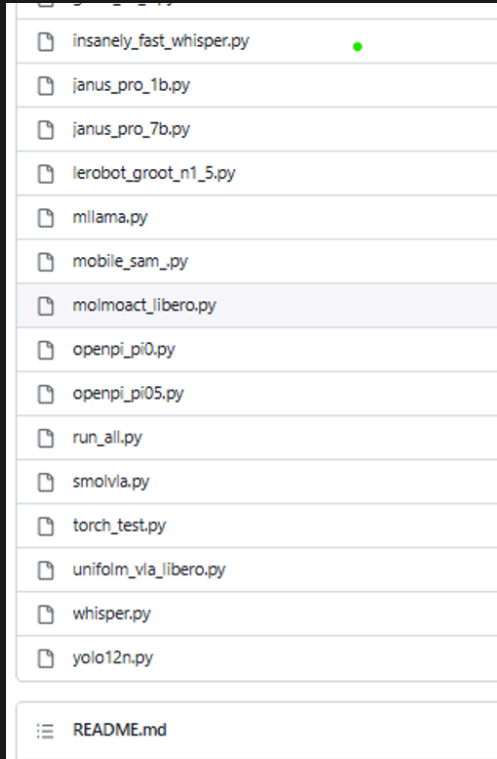
Relation with AIG & Value Add

How PAI SDK builds on — and feeds back into — the AMD AI ecosystem

Contributions — Research Desk → Vertical Software

1. PAVS transforms raw researcher code into production-ready vertical software

What we receive: Research Repo



What we receive: Research Code

```
# rocm-scripts/test/pytorch/yolo12n.py
# .....
# .....
# .....
# .....
# .....
# .....
recalled = gt_labels & detected_classes
recall = len(recalled) / len(gt_labels)
sanity_pass = recall >= RECALL_THRESHOLD
assert sanity_pass, (
    f"recall {recall:.0%} < {RECALL_THRESHOLD:.0%}")

return lats, {
    "architecture": "CNN",
    "task": "object_detection",
    "num_detections": num_detections,
    "recall": round(recall, 4),
}
if __name__ == "__main__":
    sys.exit(run(run_benchmark))
```

Single script, no structure, no device flows

What we deliver: Vertical Software

```
examples/yolov12/
├── Makefile
├── METRICS_TABLE.md
├── README.md
├── app/
│   ├── routes/
│   ├── services/
│   └── main.py & config.py
├── scripts/
│   ├── benchmark.py
│   ├── export_onnx.py
│   ├── profile_model.py
│   ├── run_val.py
│   └── update_metrics_table.py
├── tests/
│   ├── test_api.py
│   ├── test_docker.py
│   ├── test_migraphx.py
│   └── test_yolo_workflow.py
├── weights/
├── datasets/
├── requirements.txt
├── requirements_cpu.txt
└── requirements_npu.txt
```

App, scripts, tests, per-device requirements — production-ready

Flat list of scripts — no structure, no device flows, no Make targets

Physical AI & Vertical Software: Researcher's Desk → Vertical Software — same commands, every device, production-ready.

Contributions(contd.) — Building the AMD AI Ecosystem

2. AIG vs PAVS — Functional Correctness over Operator Validation

Approach	Operator validation	Functional correctness
Inputs	Dummy / synthetic tensors	Real input datasets
Measures	Throughput on isolated ops	End-to-end accuracy + throughput + latency
Pipeline	Op-level checks	Full inference pipeline (pre → infer → post)

PAVS implements the **end-to-end working pipeline** and ensures the model produces **expected outputs from real input datasets** — not just numbers from dummy inputs.

3. Developer-Friendly Inference Tools & LLM Infra

Profilers + LLM serving stacks — with tutorials for ease of adoption:

Category	Tools
Profilers	Lemonade, NPU AI Analyzer, ROCProfiler
LLM Infra	vLLM (high-throughput serving), Llama.cpp (edge inference), FastFlowLM (LLMs on NPU)

Bridging the **open-source ecosystem** ↔ **AMD HW ecosystem** — making these tools easy to use across the AMD stack.

4. Vertical Application Platform

A platform that **evolves with the client** — solving, fixing, and growing together in targeted domains:

Domain	Focus
Healthcare	Medical imaging, diagnostics
Automotive	Perception, safety systems
Industrial	Inspection, anomaly detection

Contributions(contd.) — AIG Model → Task-Specific

5. Operationalizing AIG Models for Real Applications

From Generic to Task-Specific

Took generic AIG Hub models and extended them with operation enablement, hyperparameter tuning, and backend exploration to reach application-level performance:

Backend	Role
PyTorch	Fallback path — handles missing/broken operators
torch.compile	Additional optimization on top of PyTorch
ONNX	Stable cross-device flow (CPU ↔ GPU ↔ NPU)
MIGraphX	2× improvement over torch+ROCm

MobileSAM · CenterPoint · YOLOv12 · YOLO26 — integrated into AARTS robotics pipelines, consumed by the robotics team today.

SmolVLA: Tuned for the Task

Latency	48 ms	30 ms
ROCm version	—	7.2.4
Customized to task	✗	✓

37% latency reduction through algorithmic tuning alone — on ROCm 7.2.4, without the edge-optimized kernels in ROCm 7.13 that give 2× speedup on other models.

The gains here are purely from task-specific model and algorithm optimization — not from a newer ROCm stack.

Headroom remains: upgrading to ROCm 7.13 on top of these algorithmic gains is expected to push latency even lower.

Contributions(contd.) — Ecosystem & Co-Engineering

6. Closing the Loop: Generic Model → Application Pipeline

The Full Journey

Continuing from the previous point — we don't stop at model operationalization:

1. Pick generic model from AIG Hub
2. Make it **task-specific** (tune, operationalize)
3. **Compute accuracy** on real inputs — surface gaps
4. **Communicate gaps back to AIG** for model improvement
5. Wire model into **inference pipelines** — connecting to real applications

Taking the generic model all the way to working applications — not just benchmarks.

Co-Engineering Opportunities

PAVS × Audio SDK

- Started discussions to explore co-engineering across SDK verticals
- Shared infrastructure, tooling, and pipeline patterns

Rivian (RVT) — Automotive Stack

- Using PAI SDK models to build an **in-car monitoring system** and **voice assistant** on AMD hardware

Google — Gemma on LiteRT

- Running the **Gemma** model on **LiteRT** on AMD hardware

Contributions(contd.) — PAI SDK Platform & Edge Enablement

Infrastructure: Installer, Edge HW, and LLM Stack

Installation & Edge Hardware

- **Single installer/uninstaller** for ROCm + RyzenAI in PAI SDK — one command to set up or tear down the full stack
- Extended and maintained ROCm + RAI support for **edge HW** — required targeted patches for hardware-specific gaps
- **Yocto & LTS support** — enabling model deployment on embedded edge devices

LLM Infrastructure on AMD Stack

- Integrated **stable vLLM** — high-throughput serving with quantized LLM variants on iGPU
- Integrating **FLM framework** — running LLMs on NPU
- Bridging the open-source LLM ecosystem onto the AMD edge stack

Full-stack edge enablement: installer → hardware patches → quantized LLM serving → NPU inference → Yocto deployment.

SW Stack (For Reference)

